# CHAPTER 16

## PERCOLATION AND NETWORK RESILIENCE

*A discussion of one of the simplest of processes taking place on networks, percolation, and its use as a model of network resilience*

**T**HE ULTIMATE goal in studying networks is to better understand the behavior of the systems networks represent. For instance, we study the structure of the Internet to understand better how Internet traffic flows or why communications protocols function the way they do or how we could change or rearrange the network to make it perform better. We study biochemical networks like metabolic networks because we hope they will lead to an understanding of the complex chemical processes taking place in the cell or perhaps to algorithmic tools that can help us extract biological insights from the large volumes of data generated by modern laboratory techniques.

Studies of the *structure* of networks, such as those discussed in the previous chapters of this book, are only one step towards this kind of understanding. Another important step is to make the connection between network structure and function: once we have measured and quantified the structure of a network, how do we turn the results into predictions or conclusions about how the overall system will behave? Unfortunately, progress in this area has been far slower than progress on characterizing structure, which is why a majority of this book is devoted to the discussion of structure. Nonetheless, there are some areas in which substantial progress has been made and illuminating theories and models developed. Among these are studies of network failure and resilience, of dynamical systems on networks, and of epidemic and other spreading processes. The remaining chapters of this book are devoted to a description of our current understanding of these and similar network processes. We begin in this chapter with a study of one of the simplest of network processes, percolation, which leads to an elegant theory of the robustness of networked systems to the failure of their components.

## 16.1 PERCOLATION

Imagine taking a network and removing some fraction of its vertices, along with the edges connected to those vertices—see Fig. 16.1. This process is called *percolation* (or, more precisely, *site percolation*—see below), and can be used as a model of a variety of real-world phenomena. The failure of routers on the Internet, for instance, can be formally represented by removing the corresponding vertices and their attached edges from a network representation of the Internet. In fact, about 3% of the routers on the Internet are non-functional for one reason or another at any one time, and it is a question of some practical interest what effect this will have on the performance of the network. The theory of percolation processes can help us answer this question.

Another example of a percolation phenomenon is the vaccination or immunization of individuals against the spread of disease. As discussed in Chapter 1, and at greater length in Chapter 17, diseases spread through populations over the networks of contacts between individuals. But if an individual is vaccinated against a disease and therefore cannot catch it, then that individual does not contribute to the spread of the disease. Of course, the individual is still present in the network, but, from the point of view of the spread of the disease, might as well be absent, and hence the vaccination process can again be formally represented by removing vertices.

One can see immediately that percolation processes can give rise to some interesting behaviors. The vaccination of an individual in a population, for example, not only prevents that individual from becoming infected but also prevents them from infecting others, and so has a "knock-on" effect in which the benefit of vaccinating one individual is felt by more than one. As we will show, this knock-on effect means that in some cases the vaccination of a relatively small fraction of the population can effectively prevent the spread of disease to anyone, an outcome known as *herd immunity*.

Similar effects crop up in our Internet example, although in that case they are usually undesirable. The removal or failure of a single router on the Internet prevents that router from receiving data, but also prevents data from reaching others via the failed one, forcing traffic to take another route—possibly longer or more congested—or even cutting off some portions of the network altogether. One of the goals of percolation theory on networks is to understand how the knock-on effects of vertex removal or failure affect the network as a whole.

Sometimes it is not the vertices in the network that fail but the edges. For instance, communication lines on the Internet can fail, disconnecting routers from one another, even though the routers themselves are still functioning perfectly. Phenomena like this can be modeled using a slightly different percolation process in which edges rather than vertices are removed from the appropriate network. If we need to distinguish between the two types of percolation process we could refer to them as vertex percolation on the one hand and edge percolation on the other, but in fact they are more commonly called *site percolation* and *bond percolation*, a nomenclature that derives from studies of percolation on low-dimensional lattices in physics and mathematics.[242] In this chapter we will focus principally on site percolation (i.e., removal of vertices) but bond percolation (removal of edges) will become important in Chapter 17 when we look at epidemic processes.

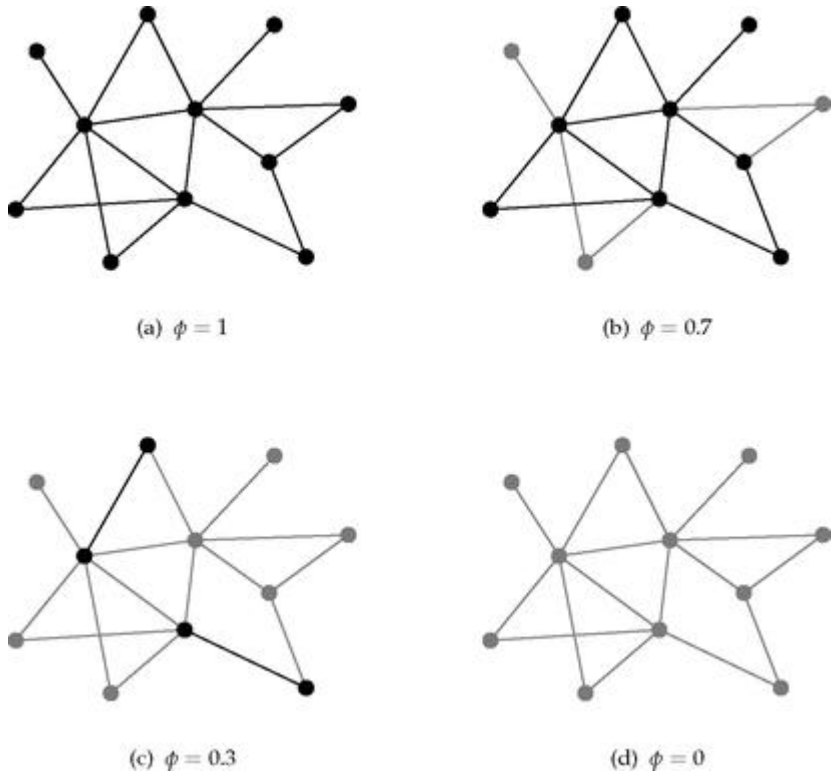



**Figure 16.1: Percolation.** A depiction of the site percolation process on a small network for various values of the occupation probability $\varphi$. Gray denotes vertices that have been removed, along with their associated edges, and black denotes those that are still present. The networks in panels (a) and (b) are above the percolation threshold while those in panels (c) and (d) are below it.

There is more than one way in which vertices can be removed from a network. In the simplest case they could be removed purely at random: we could for example take away some specified fraction of the vertices chosen uniformly at random from the entire network. This is the most commonly studied form of site percolation, and indeed for many people the word "percolation" refers specifically to this particular process. But there are many other ways in which vertices could be removed and "percolation" as used in this chapter is considered to include all of them. One popular alternative removal scheme is to remove vertices according to their degree in some fashion. For instance, we could remove vertices in order of degree from highest to lowest, an approach that turns out to make an effective vaccination strategy for the control of disease. Other approaches have also been considered occasionally, such as removing vertices with high betweenness centrality. Let us begin, however, by examining the simplest case of uniformly random removal.

## 16.2 UNIFORM RANDOM REMOVAL OF VERTICES

Consider a network in which some fraction of the vertices, selected uniformly at random, are removed. As discussed above, in many real-world situations "removal" does not imply actual physical removal of the vertices, but only that they are non-functional in some way, such as routers that have failed on the Internet, or vaccinated individuals in a network of disease-causing contacts.

Traditionally the percolation process is parametrized by a probability $\varphi$, which is probability that a vertex is present or functioning in the network. In the parlance of percolation theory, one says that the functional vertices are *occupied* and $\varphi$ is called the *occupation probability*. Thus $\varphi = 1$ indicates that all vertices in the network are occupied (i.e., no vertices have been removed) and $\varphi = 0$ indicates that no vertices are occupied (i.e., all of them have been removed).[243]

Now look again at Fig. 16.1 and consider panel (a), in which $\varphi = 1$, all vertices are present or occupied, and all vertices are connected together into a single component. (The network could have more the one component, but in this example it has only one.) Now look at the other panels. In panel (b) a few vertices have been removed, but those that remain are all still connected together by the remaining edges. In panel (c) still more vertices have been removed, and now so many are gone that the remaining vertices are no longer all connected together, having split into two small components. In the final panel, panel (d), all vertices have been removed and there is no network left at all.

The behavior we see in this small example is typical of percolation processes. When $\varphi$ is large the vertices tend to be connected together, forming a giant component that fills most of the network (although there may be small components also). But as $\varphi$ is decreased there comes a point where the giant component breaks apart and we are left only with small components. Conversely, if we increase $\varphi$ from zero we first form small components, which then grow in size and eventually coalesce to form a giant component that fills a large fraction the network.

The formation or dissolution of a giant component in this fashion is called a *percolation transition*. When the network contains a giant component we say that it *percolates* and the point at which the percolation transition occurs is called the *percolation threshold*.

The percolation transition is similar in many ways to the phase transition in the Poisson random graph at which a giant component forms (see Section 12.5). In the random graph we vary not the fraction of occupied vertices but the probability of connection between those vertices. In both cases, however, when enough of the network is removed the giant component is destroyed and we are left with only small components.

In studies of percolation the "components" that remain after vertices have been removed are in fact usually called *clusters*, another term inherited from the physics and mathematics literature and one that we will use here—it will be useful to distinguish between the "components" of the underlying network and the "clusters" of the percolation process. That is, we will use "component" to refer to connected groups of vertices on the original network before any vertices have been removed and "cluster" to refer to those after removal. The giant component of the percolation process, if there is one, is thus properly called the *giant cluster*.[244]

The percolation transition plays a central role in our interpretation of percolation phenomena. In a network like the Internet, for example, there has to be a giant cluster if the network is to perform its intended function as a communications network. If the network has only small clusters, as in Fig. 16.1c, then every vertex has a connection to, at most, a handful of others and is cut off from everyone else. If there is a giant cluster, on the other hand, then the members of that giant cluster, who are a finite fraction of all vertices in the network, are connected and can communicate with one another, although the remainder of the network is still cut off. Thus the presence of a giant cluster is an indicator of a network that is at least partly performing its intended function, while the size of the giant cluster tells us exactly how much of the network is working.

### 16.2.1 UNIFORM REMOVAL IN THE CONFIGURATION MODEL

To gain some understanding of the percolation transition and the giant cluster, let us consider the behavior of the site percolation process on networks generated using the configuration model of Chapter 13, a simple but useful model of a network with a specified degree distribution. We can calculate the properties of the giant percolation cluster in the configuration model by a method similar to the one we used for the giant component of configuration model in Section 13.8.

Consider a configuration model network with degree distribution $p_k$ and a percolation process on that network in which vertices are present or occupied with occupation probability $\varphi$ as above. Now consider one of the vertices that is present in the network (i.e., one that has not been removed). If that vertex is to belong to the giant cluster it must be connected to it via at least one of its neighbors. Equivalently, it is not a member of the giant cluster if and only if it is not connected to the giant cluster via any of its neighbors. Following the notation of Section 13.8, let us define $u$ to be the average probability that a vertex is not connected to the giant cluster via a particular neighbor. Then if the vertex in question has degree $k$, the total probability of its not belonging to the giant cluster is $u^k$ . And if we then average over the probability distribution $p_k$ of the degree we find that the average probability of not being in the giant cluster is $\sum_k p_k u^k = g_0(u)$, where

$$g_0(z) = \sum_{k=0}^{\infty} p_k z^k$$

(16.1)

is the generating function for the degree distribution, as defined previously in Eq. (13.48). Then the average probability that a vertex *does* belong to the giant cluster is $1 - g_0(u)$.

Bear in mind, however, that this is for a vertex that is itself assumed not to have been removed from the network. Vertices that have been removed are obviously not members of the giant cluster either. Thus out of all the original vertices in the network the total fraction $S$ that are in the giant cluster is equal to the fraction $\varphi$ that have not been removed times the probability $1 - g_0(u)$ that they are in the giant cluster:

$$S = \phi[1 - g_0(u)].$$

(16.2)

We still need to calculate the value of $u$, which is the average probability that a vertex is not connected to the giant cluster via a particular neighboring vertex. There are two ways to not be connected to the giant cluster via a neighbor: either the neighbor in question—let us call it vertex A—has been removed, which happens with probability $1 - \varphi$, or it is present (probability $\varphi$) but it is not itself a member of the giant cluster. The latter happens if A is not connected to the giant cluster via any of its other neighbors. Suppose there are $k$ of these. Then the probability that none of them connects us to the giant cluster is $u^k$. Adding everything together, the total probability that

we are not connected to the giant cluster via A is $1 - \varphi + \varphi u^k$.

Since A is reached by following an edge, the value of $k$ in this case is distributed according to the excess degree distribution

$$q_k = \frac{(k+1)p_{k+1}}{\langle k \rangle},$$

(16.3)

(see Section 13.3) where $\langle k \rangle$ is the average degree in the network. Averaging over this distribution, we then arrive at an expression for the average probability $u$ thus:

$$\begin{aligned} u &= \sum_{k=0}^{\infty} q_k (1 - \phi + \phi u^k) = 1 - \phi + \phi \sum_{k=0}^{\infty} q_k u^k \\ &= 1 - \phi + \phi g_1(u), \end{aligned}$$

(16.4)

where

$$g_1(z) = \sum_{k=0}^{\infty} q_k z^k$$

(16.5)

is the generating function for the excess degree distribution, defined previously in Eq. (13.49), and we have made use of the normalization condition $\sum_k q_k = 1$.

Equations (16.2) and (16.4) give us a complete solution for the size of the giant cluster in our network.[245] In practice it is often not possible to solve Eq. (16.4) in closed form, but there is an elegant graphical representation of the solution as follows.

Consider Fig. 16.2a, which gives a sketch of the form of the function $g_1(u)$. The exact form of the curve will depend on the degree distribution, but we know the general shape: $g_1$ is a polynomial with all coefficients non-negative (because they are probabilities), so it must have a non-negative value and all derivatives non-negative for $u \geq 0$. Thus in general it is an increasing function of $u$ and curves upward as shown in the figure.

To get the function $1 - \varphi + \varphi g_1(u)$ that appears on the right-hand side of Eq. (16.4) we first multiply $g_1(u)$ by $\varphi$ then add $1 - \varphi$. Graphically that is equivalent to compressing the unit square of Fig. 16.2a (along with the curve it contains) until it has height $\varphi$ and then shifting it upward a distance $1 - \varphi$ as shown in Fig. 16.2b. The point or points at which the resulting curve crosses the line $y = u$ (dotted line in Fig. 16.2b) are then the solutions to Eq. (16.4).

In Fig. 16.2b there are two such solutions. One is a trivial solution at $u = 1$. This solution always exists because $g_1(1) = 1$ for any correctly normalized excess degree distribution $q_k$. But there is also a non-trivial solution with $u < 1$, indicated by the dot in the figure. Only if we have such a

non-trivial solution can there be a giant cluster in the network and the value of $u$ for this solution gives us the size of the giant cluster via Eq. (16.2). (The $u = 1$ solution gives $S = 0$ in Eq. (16.2) and so doesn't give us a giant cluster.)

Now consider Fig. 16.2d, which shows the equivalent graphical solution of Eq. (16.4) for a smaller value of $\varphi$. Now the curve of the generating function has been compressed more and the result is that the non-trivial solution for $u$ has vanished. Only the trivial solution at $u = 1$ remains and so in this regime there can be no giant cluster.

Figure 16.2c shows the borderline case between cases (b) and (d). The nontrivial solution for $u$ vanishes at the point shown, where the curve just meets the dotted line. Mathematically this is the point at which the curve is tangent to the dotted line at $u = 1$, i.e., the point where its gradient at $u = 1$ is 1. In other words the percolation threshold occurs when

$$\left[\frac{\mathrm{d}}{\mathrm{d}u}\left(1 - \phi + \phi g_1(u)\right)\right]_{u=1} = 1.$$

(16.6)

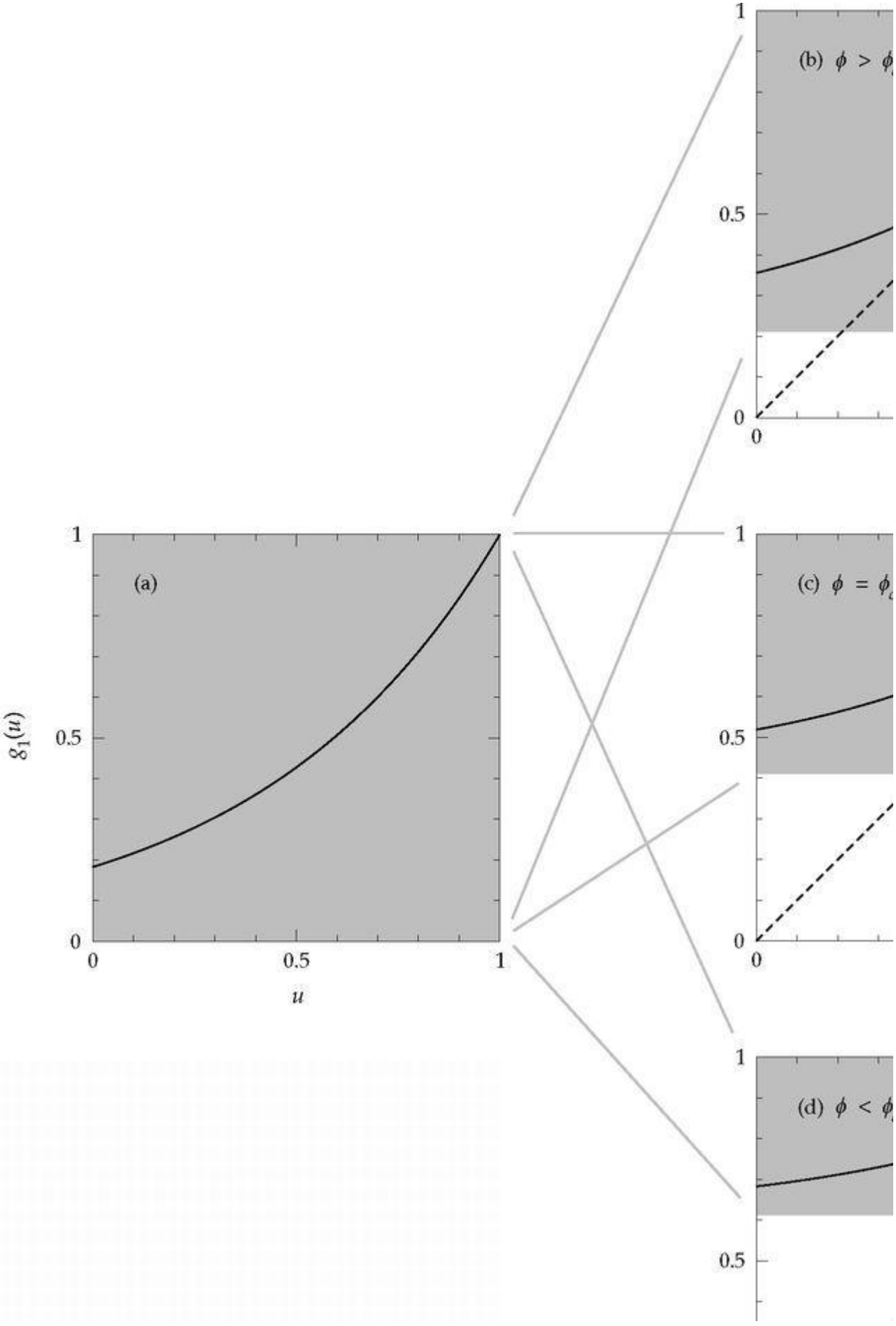

(a)
(b) $\phi > \phi_c$
(c) $\phi = \phi_c$
(d) $\phi < \phi_c$
$g_1(u)$
$u$
0
0.5
1

**Figure 16.2: Graphical solution of Eq. (16.4).** The generating function $g_1(u)$ for the excess degree distribution, panel (a), is compressed by a factor of $\varphi$ and shifted upward to give the functional form $y = 1 - \varphi + \varphi g_1(u)$. The resulting curve is shown for three different values of $\varphi$ in panels (b), (c), and (d). In panel (b) $\varphi$ is sufficiently large that there is a nontrivial solution where the curve crosses the dotted line $y = u$. In panel (d) $\varphi$ is smaller and there is only a trivial solution at $u = 1$. Panel (c) shows the borderline case where the curve is tangent to the dotted line at $u = 1$.

Performing the derivative we then find that the value of $\varphi$ at the transition, which we call the *critical value*, denoted $\varphi_c$, is

$$\phi_c = \frac{1}{g_1'(1)}.$$

(16.7)

We can express the critical value more directly in terms of the degree distribution by making use of the definitions of the generating function $g_1$ and the excess degree distribution, Eqs. (16.3) and (16.5). Substituting one into the other and differentiating, we find that

$$\begin{aligned} g_1'(1) &= \frac{1}{\langle k\rangle}\sum_{k=0}^{\infty} k(k+1)p_{k+1} = \frac{1}{\langle k\rangle}\sum_{k=0}^{\infty} k(k-1)p_k \\ &= \frac{\langle k^2\rangle - \langle k\rangle}{\langle k\rangle}, \end{aligned}$$

(16.8)

and hence the critical occupation probability $\varphi_c$ is given by

$$\phi_c = \frac{\langle k\rangle}{\langle k^2\rangle - \langle k\rangle},$$

(16.9)

an expression first given by Cohen *et al.* [74].

This equation tells us the minimum fraction of vertices that must be present or occupied in our configuration model network for a giant cluster to exist. Thus, for instance, if we were to consider the configuration model as a simple model of the Internet, we would want to make $\varphi_c$ low, so that the network will have a giant cluster even when some fraction of vertices are non-functional, and hence go on functioning as a communication network. We can arrange this by making sure that

$\langle k^2 \rangle \gg \langle k \rangle$ for the network. If, for instance, the network had a Poisson degree distribution,

$$p_k = e^{-c} \frac{c^k}{k!},$$

(16.10)

where $c$ is the mean degree, then $\langle k \rangle = c$ and $\langle k^2 \rangle = c(c + 1)$, so

$$\phi_c = \frac{1}{c}.$$

(16.11)

Then if we can make $c$ large we will have a network that can withstand the loss of many of its vertices. For $c = 4$, for example, we would have $\phi_c = \frac{1}{4}$, meaning that $\frac{3}{4}$ of the vertices would have to fail before the giant cluster is destroyed. A network that can tolerate the loss of a large fraction of its vertices in this way is said to be *robust* against random failure.

The degree distribution of the Internet, however, is not Poissonian. In fact, as discussed in Section 8.4, the Internet's degree distribution appears roughly to follow a power law with an exponent $\alpha \simeq 2.5$ (see Table 8.1). As we showed in Section 8.4.2, power laws with exponents in the range $2 < \alpha < 3$, which includes most real-world examples, have a finite mean $\langle k \rangle$, but their second moment $\langle k^2 \rangle$ diverges. In this case Eq. (16.9) implies that $\varphi_c = 0$. In other words, no matter how many vertices we remove from the network there will *always* be a giant cluster. Scale-free networks—those with power-law degree distributions—are thus highly robust networks that can survive the failure of any number of their vertices, a point first highlighted in the work of Albert *et al.* [14].

In practice, as discussed in Section 8.4.2, the second moment of the degree distribution is never actually infinite in any finite network. Even for finite $n$ though it can still become very large, which can result in non-zero but very small values of $\varphi_c$, so that the network is still highly robust.

The structure of the real Internet is not the same as that of a configuration model with the same degree distribution. It has all sorts of layers and levels of structure engineered into it, as discussed in Section 2.1. Nonetheless, it does appear to be quite robust to random removal of its vertices. For instance, Albert *et al.* [14] simulated the behavior of the Internet as vertices were randomly removed from its structure and found that performance is hardly affected at all by the removal of even a significant fraction of vertices. (Performance is of course completely destroyed for the vertices that are themselves removed, but for the remaining ones the effects are relatively minor.) These and related results are discussed further in Section 16.3.

Network robustness also plays an important role in the vaccination example mentioned at the start of the chapter. A disease spreading over a contact network between individuals can only reach a significant fraction of the population if there is a giant cluster in the network. If the network contains only small clusters then an outbreak of the disease will be hemmed in by vaccinated individuals and unable to spread further than the small cluster in which it starts. Thus one does not have to vaccinate the entire population to prevent disease spread. One need only vaccinate enough of them to bring the network below its percolation threshold. This is the herd immunity effect mentioned earlier.

In this example, network robustness is a bad thing. The fewer individuals we have to vaccinate to destroy the giant cluster the better. Thus small values of $\varphi_c$ are bad in this case and large values are good. Unfortunately, we usually don't have much control over the degree distributions of contact networks, so we may be stuck with a low value of $\varphi_c$ whether we like it or not. In particular, if the network in question has a power-law (or approximately power-law) degree distribution, then $\varphi_c$ may be very small, implying that almost all vertices have to be vaccinated to wipe out the disease. Some contact networks do indeed appear to have roughly power-law degree distributions [167, 197, 198] and it may be very difficult to eradicate some diseases as a result [264].

It is interesting to ask how the special behavior of power-law networks shows up in the graphical solution of Fig. 16.2. The answer is that, since $g'_1(1)$ is infinite in the power-law case (because $\langle k^2 \rangle$ diverges in Eq. (16.8) while $\langle k \rangle$ remains finite), the curve of $g_1(u)$ has infinite slope at $u = 1$. Thus $g_1(u)$ must look something like Fig. 16.3. Because of the infinite slope, it makes no difference how much we compress the function (as in Fig. 16.2)—the curve will always drop below the line of $y = u$ before coming back up again and crossing it to give a non-trivial solution for $u$.

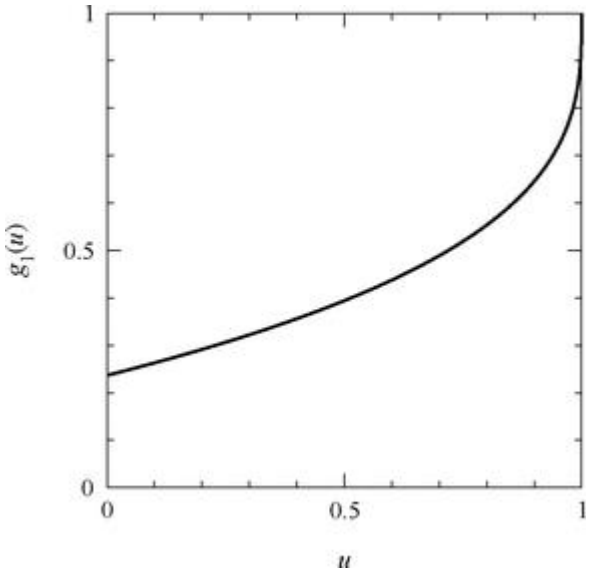



**Figure 16.3: Generating function for the excess degree distribution in a scale-free network.** The generating function $g_1(u)$ for a network with a power-law degree distribution has a derivative that diverges as $u \to 1$, though the value of the generating function remains finite and tends to 1 in this limit. Thus the function looks generically like the curve sketched here.

The position of the percolation threshold is not the only quantity important in assessing the robustness of a network. The size of the giant cluster also plays a role because it tells us what fraction of the network will be connected and functional. To find the size of the giant cluster we need to solve Eq. (16.4) for $u$ and then substitute the result back into Eq. (16.2). In many cases, as we have said, we cannot solve for $u$ exactly, but in some cases we can. Consider, for example, a network with an exponential degree distribution given by

$$p_k = (1 - \mathrm{e}^{-\lambda})\,\mathrm{e}^{-\lambda k},$$

(16.12)

where $\lambda > 0$ and the leading factor of $1 - e^{-\lambda}$ insures that the distribution is properly normalized. Then, as shown in Section 13.9.2, we have

$$g_0(z) = \frac{e^\lambda - 1}{e^\lambda - z}, \qquad g_1(z) = \left(\frac{e^\lambda - 1}{e^\lambda - z}\right)^2,$$

(16.13)

and Eq. (16.4) becomes

$$u(e^\lambda - u)^2 - (1 - \phi)(e^\lambda - u)^2 - \phi(e^\lambda - 1)^2 = 0.$$

(16.14)

This is a cubic equation, which is ugly (though not impossible) to solve. In this case, however, we don't have to solve it directly. We observe instead that $u = 1$ is always a solution of Eq. (16.4) and hence that our cubic equation must contain a factor of $u - 1$. A few moments work reveals that indeed this is the case. Equation (16.14) factorizes as

$$(u - 1)\left[u^2 + (\phi - 2e^\lambda)u + \phi - 2\phi e^\lambda + e^{2\lambda}\right] = 0.$$

(16.15)

Thus the two other solutions for $u$ satisfy the quadratic equation

$$u^2 + (\phi - 2e^\lambda)u + \phi - 2\phi e^\lambda + e^{2\lambda} = 0.$$

(16.16)

Of these two solutions one is greater than one for $\lambda > 0$ and so cannot be our probability $u$. The other is

$$u = e^\lambda - \tfrac{1}{2}\phi - \sqrt{\tfrac{1}{4}\phi^2 + \phi(e^\lambda - 1)}.$$

(16.17)

Now we can plug this value back into Eq. (16.2) to get an expression for the size of the giant cluster as a fraction of the whole network:

$$\begin{aligned} S &= \phi\left[1 - \frac{2(e^\lambda - 1)}{\phi + \sqrt{\phi^2 + 4\phi(e^\lambda - 1)}}\right] \\ &= \phi\left[1 - 2(e^\lambda - 1)\frac{\phi - \sqrt{\phi^2 + 4\phi(e^\lambda - 1)}}{\phi^2 - (\phi^2 + 4\phi(e^\lambda - 1))}\right] \\ &= \tfrac{3}{2}\phi - \sqrt{\tfrac{1}{4}\phi^2 + \phi(e^\lambda - 1)}. \end{aligned}$$

(16.18)

Notice that the solution for $u$, Eq. (16.17), can become greater than 1 for sufficiently small $\varphi$, which is unphysical. In this regime the only acceptable solution is the trivial $u = 1$ solution, which gives $S = 0$ and so there is no giant cluster when this happens. This gives us an alternative way to derive the position of the percolation transition. The transition takes place at the point where Eq. (16.17) equals one, i.e., when

$$e^\lambda - 1 - \tfrac{1}{2}\phi = \sqrt{\tfrac{1}{4}\phi^2 + \phi(e^\lambda - 1)}.$$

(16.19)

Squaring both sides and rearranging for $\varphi$ we find that the percolation threshold falls at

$$\phi_c = \tfrac{1}{2}(e^\lambda - 1).$$

(16.20)

It is left as an exercise to demonstrate that this is the same result we get if we apply the general formula, Eq. (16.7).

Note also that if $\lambda$ becomes sufficiently large then the value of $\varphi_c$ given by Eq. (16.20) can become greater than one. For values of $\lambda$ this large there is no percolation transition and the system never percolates because $\varphi$ can never be greater than $\varphi_c$. The value of $\lambda$ at which we enter this regime is the value at which $\frac{1}{2}(e^\lambda - 1) = 1$, which gives $\lambda = \ln 3$. Upon closer inspection, it turns out that this is precisely the point at which the network itself loses its giant *component*,[246] which explains why percolation is not possible beyond this point. For $\lambda > \ln 3$ the network has no giant

component, and hence it is not possible to have a giant cluster even if every vertex in the network is present. (A similar result of course applies to all networks—a giant percolation cluster is never possible in a network without a giant component.)

Figure 16.4 shows a plot of the value of $S$ for our exponential network with $\lambda = \frac{1}{2}$ as a function of $\varphi$. For small $\varphi$ there is a region in which there are only small clusters and no giant cluster. When we pass through the percolation transition, marked by the dotted line in the figure, a giant cluster appears and grows smoothly from zero as $\varphi$ increases. This is an example of what a physicist would call a continuous phase transition.[247] We saw other examples in Sections 13.9 and 15.2.5.

The overall behavior shown in Fig. 16.4 is typical of percolation in networks. For most degree distributions we expect $S$ to take a similar form with a continuous phase transition, as we can demonstrate by the following argument. Suppose the generating function $g_1(u)$ is well-behaved near $u = 1$, having all its derivatives finite,[248] then we can expand it about this point as

$$\begin{aligned} g_1(u) &= g_1(1) + (u-1)g_1'(1) + \tfrac{1}{2}(u-1)^2 g_1''(1) + \mathrm{O}(u-1)^3 \\ &= 1 + \frac{u-1}{\phi_c} + \tfrac{1}{2}(u-1)^2 g_1''(1) + \mathrm{O}(u-1)^3, \end{aligned}$$

(16.21)

where we have made use of $g_1(1) = 1$ (see Eq. (13.20)) and Eq. (16.7). Substituting into Eq. (16.4), we then find that

$$u = 1 + \frac{\phi}{\phi_c}(u-1) + \tfrac{1}{2}\phi(u-1)^2 g_1''(1) + \mathrm{O}(u-1)^3$$

(16.22)

or

$$u - 1 = \frac{2}{g_1''(1)} \frac{\phi_c - \phi}{\phi_c \phi} + \mathrm{O}(u-1)^2.$$

(16.23)

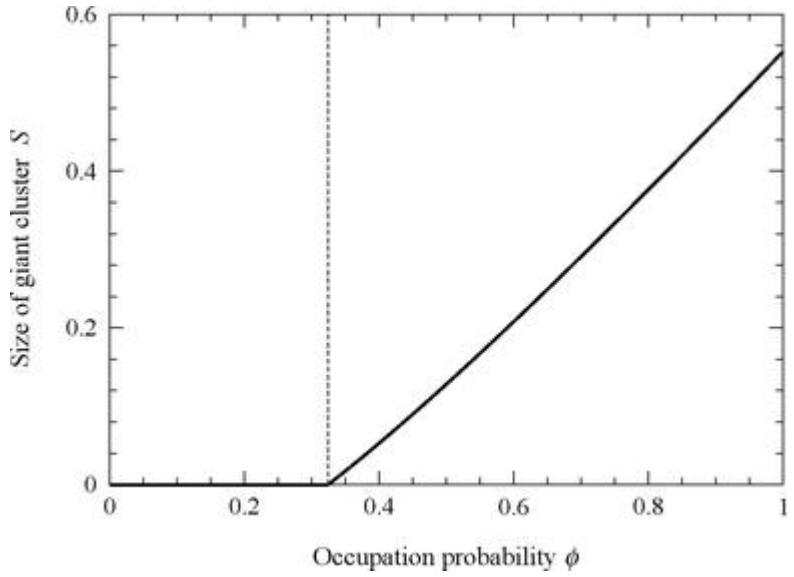



**Figure 16.4: Size of the giant cluster for site percolation in the configuration model.** The curve indicates the size of the giant cluster for a configuration model with an exponential degree distribution of the form (16.12) with $\lambda = \frac{1}{2}$, as given by Eq. (16.18). The dotted line indicates the position of the percolation transition, Eq. (16.20).

We can similarly expand $g_0(u)$ as

$$\begin{aligned} g_0(u) &= g_0(1) + (u-1)g_0'(1) + \mathrm{O}(u-1)^2 \\ &= 1 + \frac{2\langle k\rangle}{g_1''(1)}\,\frac{\phi_c - \phi}{\phi_c \phi} + \mathrm{O}(\phi - \phi_c)^2, \end{aligned}$$

(16.24)

where we have used $g_0(1) = 1$ and Eqs. (13.22) and (16.23). Substituting into Eq. (16.2) then gives us

$$S = \frac{2\langle k\rangle}{g_1''(1)}\,\frac{\phi - \phi_c}{\phi_c} + \mathrm{O}(\phi - \phi_c)^2,$$

(16.25)

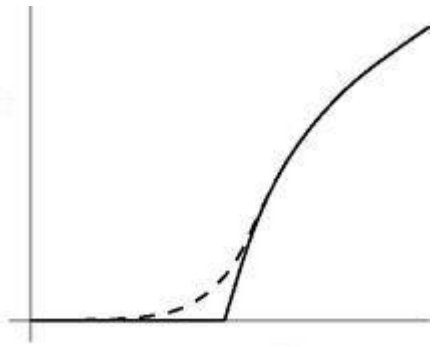
In other words, $S$ varies linearly with $\varphi - \varphi_c$ just above the percolation transition, going to zero continuously as we approach the transition from above. Thus we would expect the percolation transition for essentially all degree distributions to look generically like the curve in Fig. 16.4, with

a continuous phase transition as we pass the percolation threshold.[249]

This result is important, because it implies that the giant cluster becomes very small as we approach the percolation transition from above. In other words, the network may be "functional" in the sense of having a giant cluster, but the functional portion of the network is vanishingly small. If the network is a communication network, for example, then a finite fraction of all the vertices in the network can communicate with one another so long as there is a giant cluster, but that fraction becomes very small as we approach the percolation threshold, meaning that in practice most vertices are cut off. Thus one could argue that it is misleading to interpret the percolation threshold as the point where the network stops functioning: in effect most of it has stopped functioning before we reach this point. To fully describe the functional state of the network one should specify not only whether it contains a giant cluster but also what the size of that cluster is.

It is also important to note that the sharp percolation transition of Fig. 16.4 is only truly seen in an infinite network. For networks of finite size—which is all real networks, of course—the transition gets rounded off. To see this, consider the behavior of the giant cluster in a finite-sized network. Technically, in fact, there *is* no giant cluster for an individual finite network. The proper definition of the giant cluster, like the giant component in a random graph, is as a cluster whose size scales in proportion to the size of the network (see Section 12.5). But it makes no sense to talk about the scaling of a cluster with network size when the size of the network is fixed. In practice, therefore, we normally consider instead the largest cluster, which is a reasonable proxy for the giant cluster in a finite-size network. Its size as a fraction of the size of the network should be a reasonable approximation to the size of the giant cluster given by our theory when we are above the percolation transition.

Below the transition the largest cluster will be small in size, but not zero, and hence fills a small but non-zero fraction of the network, in rough but not perfect agreement with the theoretical prediction $S = 0$. Furthermore, this non-zero value grows as we approach the transition point because small clusters in general, including the largest one, grow as the occupation probability $\varphi$ increases. The net result is a slight rounding of the sharp transition predicted by the theory, which is often visible, for example, in computer simulations of percolation on smaller networks. Effects such as this that show up only in finite-sized systems are known as *finite size effects*.



The phase transition at which the giant cluster appears is only sharp in an infinite system (solid line). In a finite sized system it gets rounded off (dashed line).

Even in the limit of large network size there are exceptions to the behavior of Fig. 16.4 and Eq. (16.25). Consider a network with a power-law degree distribution with exponent $2 < \alpha < 3$, as discussed above. In this case our assumption that the derivatives of $g_1$ are finite does not hold (see Fig. 16.3 and the accompanying discussion), so the argument above breaks down. Not only does the percolation threshold fall at $\varphi_c = 0$ for power-law networks, but the giant cluster does not grow linearly as $\varphi$ increases. In general it will grow slower than linearly, the exact functional form depending on the shape of $g_1(u)$ near $u = 1$. For example, a typical form is

$$1 - g_1(u) = c(1-u)^\beta,$$

(16.26)

near $u = 1$ with $c$ and $\beta$ positive constants. Provided $\beta < 1$ this makes the gradient of $g_1(u)$ (and all higher derivatives) infinite at $u = 1$ while still ensuring that $g_1(1) = 1$. With this form for $g_1(u)$, Eq. (16.4) implies

$$1 - u = (c\phi)^{1/(1-\beta)}.$$

(16.27)

Then

$$g_0(u) \simeq g_0(1) + g_0'(1)(u-1) = 1 + \langle k \rangle (u-1),$$

(16.28)

close to $u = 1$, with $\langle k \rangle$ finite so long as the power-law exponent $\alpha > 2$, and hence the giant cluster has size

$$S = \phi[1 - g_0(u)] \simeq \phi \langle k \rangle (1-u) \sim \phi^{(2-\beta)/(1-\beta)},$$

(16.29)

which goes to zero faster than linearly[251] as $\varphi \to 0$ since $(2-\beta)/(1-\beta) > 1$ if $\beta < 1$.

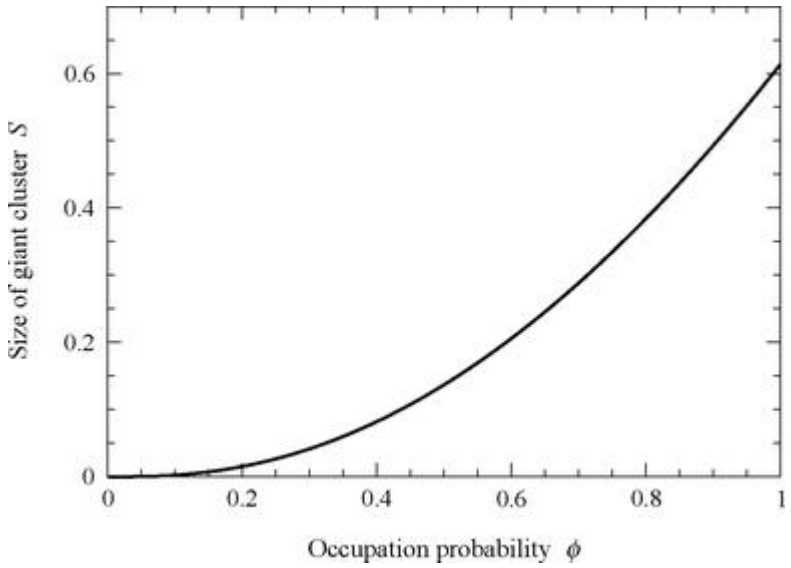



**Figure 16.5: Size of the giant cluster for a network with power-law degree distribution.** The size of the giant cluster for a scale-free configuration model network with exponent $\alpha = 2.5$, a typical value for real-world networks. Note the non-linear form of the curve near $\varphi = 0$, which means that $S$, while technically non-zero, becomes very small in this regime. Contrast this figure with Fig. 16.4 for the giant cluster size in a network with an exponential degree distribution.

Thus we expect the giant cluster to become very small as $\varphi \to 0$. Figure 16.5 shows the equivalent of Fig. 16.4 for a scale-free network with exponent $\alpha = 2.5$, derived from numerical solutions of Eqs. (16.2) and (16.4) and the non-linear form of $S$ close to $\varphi = 0$ is clear.

This result mitigates somewhat our earlier statement that scale-free networks are highly robust because $\varphi_c = 0$. It is true that the percolation threshold is zero in these networks and hence that there is a giant cluster for any positive $\varphi$, but that giant cluster can become exceedingly small. A communication network with a power-law degree distribution, for instance, might be formally functional for very small values of $\varphi$, but in practice the fraction of vertices that could communicate with one another would be so small that the network would probably not be of much use.

## 16.3 NON-UNIFORM REMOVAL OF VERTICES

In the first part of this chapter we have considered percolation phenomena in the case where vertices are removed from a network uniformly at random. This is the classical form of percolation long studied by physicists and mathematicians. When discussing networks, however, it is interesting also to consider other ways in which vertices might be removed. In Section 16.1, for example, we mentioned the possibility of removing vertices in order of their degrees, starting with the highest degrees and working down. This might be effective, for example, as a vaccination strategy for preventing the spread of disease: should they become infected, the high degree vertices in the network clearly present a disease risk to their many neighbors, so perhaps vaccinating them first would be a sensible approach.

Let us consider a generalization of our percolation process in which the occupation probability of a vertex can now depend on its degree. We define $\varphi_k$ to be the probability that a vertex with degree $k$ is present or occupied in our network. If $\varphi_k$ is a constant, independent of $k$, then we recover the uniform scenario of previous sections. On the other hand, if $\varphi_k = 1$ for all vertices with degree $k < k_0$ for some constant $k_0$, and $\varphi_k = 0$ for all vertices with $k \geq k_0$, then we effectively remove from the network all vertices with degree $k_0$ or greater. A host of other choices are also possible, resulting in more complex removal patterns.

Let us again look at percolation on configuration model networks and as before define $u$ to be the average probability a vertex is not connected to the giant cluster via one of its neighbors. If the vertex has degree $k$ then the probability that it is not connected to the giant cluster via any of its neighbors is $u^k$ and the probability that it *is* connected to the giant cluster is $1 - u^k$. But in order to belong to the giant cluster, the vertex itself must also be present, which happens with probability $\varphi_k$, so the probability of it being a member of the giant cluster is $\varphi_k (1 - u^k)$.

Now we average over the probability distribution $p_k$ of the degree to find the average probability of being in the giant cluster and get

$$\begin{aligned} S &= \sum_{k=0}^{\infty} p_k \phi_k (1 - u^k) = \sum_{k=0}^{\infty} p_k \phi_k - \sum_{k=0}^{\infty} p_k \phi_k u^k \\ &= f_0(1) - f_0(u), \end{aligned}$$

(16.30)

where

$$f_0(z) = \sum_{k=0}^{\infty} p_k \phi_k z^k.$$

(16.31)

Notice that this new generating function is not normalized in the conventional fashion—the value $f_0(1)$ that appears in Eq. (16.30) is not in general equal to one. Instead it is given by

$$f_0(1) = \sum_{k=0}^{\infty} p_k \phi_k = \overline{\phi},$$

(16.32)

which is the average probability that a vertex is occupied.

We can calculate the value of $u$ using an approach similar to that for the uniform percolation scenario. The value of $u$ is the probability that you are not connected to the giant cluster via your neighbor, which happens if either the neighbor is not occupied or if it is occupied but it is not connected to the giant cluster via any of its other neighbors. Let $k$ now be the excess degree of the neighboring vertex. Then the probability that the neighbor is not occupied is $1 - \varphi_{k+1}$. Notice that the index is $k + 1$ because $\varphi_k$ is defined in terms of the total degree of a vertex, which is one greater than the excess degree (see Section 13.3). The probability that the neighbor *is* occupied but is itself not connected to the giant cluster is $\varphi_{k+1} u^k$. Adding up the terms and averaging over the distribution $q_k$ of the excess degree, we then find that

$$u = \sum_{k=0}^{\infty} q_k (1 - \phi_{k+1} + \phi_{k+1} u^k) = 1 - f_1(1) + f_1(u),$$

(16.33)

where

$$f_1(z) = \sum_{k=0}^{\infty} q_k \phi_{k+1} z^k$$

(16.34)

and we have used $\sum_k q_k = 1$.

Like $f_0(z)$, the function $f_1(z)$ is not normalized to unity. The definition of $f_1(z)$ looks slightly odd because of the subscript $k + 1$. If we prefer we can write it using the full expression for the excess degree distribution, Eq. (16.3), which gives

$$\begin{aligned} f_1(z) &= \frac{1}{\langle k \rangle} \sum_{k=0}^{\infty} (k+1) p_{k+1} \phi_{k+1} z^k \\ &= \frac{1}{\langle k \rangle} \sum_{k=1}^{\infty} k p_k \phi_k z^{k-1}, \end{aligned}$$

(16.35)

which has a more symmetric look about it. Note also that

$$f_1(z) = \frac{f_0'(z)}{g_0'(1)},$$

(16.36)

where $g_0(z)$ is defined as before. This expression can be useful for calculating $f_1(z)$ once $f_0(z)$ has been found.

Equations (16.30) and (16.33), which were first given by Callaway *et al.* [62], give us a complete solution for the size of the giant cluster for our generalized percolation process.

As an example of their use, consider again a network with exponential degree distribution given by Eq. (16.12) and suppose we remove all vertices that have degree $k_0$ or greater. That is, we choose

$$\phi_k = \begin{cases} 1 & \text{if } k < k_0, \\ 0 & \text{otherwise.} \end{cases}$$

(16.37)

Then we have

$$f_0(z) = (1 - \mathrm{e}^{-\lambda}) \sum_{k=0}^{k_0-1} \mathrm{e}^{-\lambda k} z^k = (1 - \mathrm{e}^{-\lambda k_0} z^{k_0}) \frac{\mathrm{e}^{\lambda} - 1}{\mathrm{e}^{\lambda} - z},$$

(16.38)

and

$$\begin{aligned} f_1(z) &= \frac{f_0'(z)}{g_0'(1)} \\ &= \left[(1 - \mathrm{e}^{-\lambda k_0} z^{k_0}) - k_0 \mathrm{e}^{-\lambda(k_0-1)} z^{k_0-1} (1 - \mathrm{e}^{-\lambda} z)\right] \left(\frac{\mathrm{e}^{\lambda} - 1}{\mathrm{e}^{\lambda} - z}\right)^2. \end{aligned}$$

(16.39)

For this choice Eq. (16.33) becomes a polynomial equation of order $k_0$ and unfortunately such equations are not solvable exactly for their roots (unless $k_0 \leq 4$). It is, however, fairly easy to find the roots numerically, especially given that we know that the root of interest in this case lies in the range between zero and one, and then we can calculate the size of the giant cluster from (16.30).

Figure 16.6a shows the results of such a calculation, plotted as a function of $k_0$. Looking at this figure, consider what happens as we lower $k_0$ from an initial high value, effectively removing more and more of the high-degree vertices in our network. As the figure shows, the size of the giant cluster decreases only slowly at first. This is because there are not many vertices of very high degree in the network, so very few have been removed. Once $k_0$ passes a value around 10, however, our attack on the network starts to become evident in a shrinking of the giant cluster, which becomes progressively more rapid until the size of the cluster reaches zero around $k_0 = 5$.

One might be forgiven for thinking that Fig. 16.6a portrays a network quite resilient to the removal of even its highest-degree vertices: it appears that we have to remove vertices all the way down to degree five in order to break up the giant cluster. This impression is misleading, however, because it fails to take account of the fact that the vast majority of vertices in the network are of very low degree, so that even when we have removed all vertices with degree greater than five, we have still removed only a small fraction of all vertices.

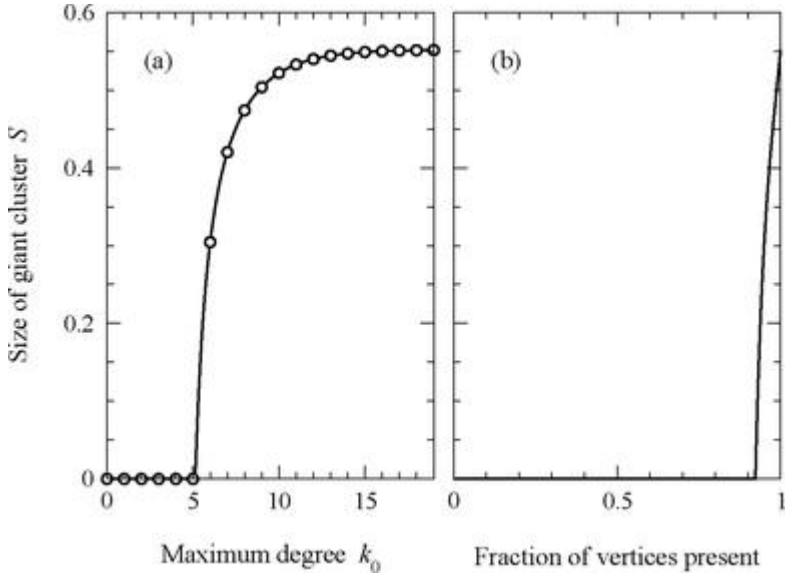



**Figure 16.6: Size of the giant percolation cluster as the highest degree vertices in a network are removed.** (a) The size of the giant cluster in a network with an exponential degree distribution $p_k \sim \mathrm{e}^{-\lambda k}$ with $\lambda = \frac{1}{z}$ as vertices are removed in order of degree, starting from those with the highest degree. The curve is shown as a function of the degree $k_0$ of the highest-degree vertex remaining in the network. Technically, since $k_0$ must be an integer, the plot is only valid at the integer points marked by the circles; the curves are just an aid to the eye. (b) The same data plotted now as a function of the fraction $\overline{\phi}$ of vertices remaining in the network.

Perhaps a more useful representation of the solution is to plot it as a function of the fraction $\overline{\phi}$ of occupied vertices in the network, which is

$$\overline{\phi} = f_0(1) = 1 - \mathrm{e}^{-\lambda k_0}.$$

(16.40)

Figure 16.6b shows the result replotted in this way and reveals that the giant cluster in fact disappears completely when only about 8% of the highest-degree vertices in the network have been removed. By contrast, when we removed vertices uniformly at random, as shown in Fig. 16.4, we had to remove nearly 70% of the vertices to destroy the giant cluster. Though the difference is startling, however, it is also intuitively reasonable. The high-degree vertices have a lot of connections, all of which are lost if we remove those vertices.

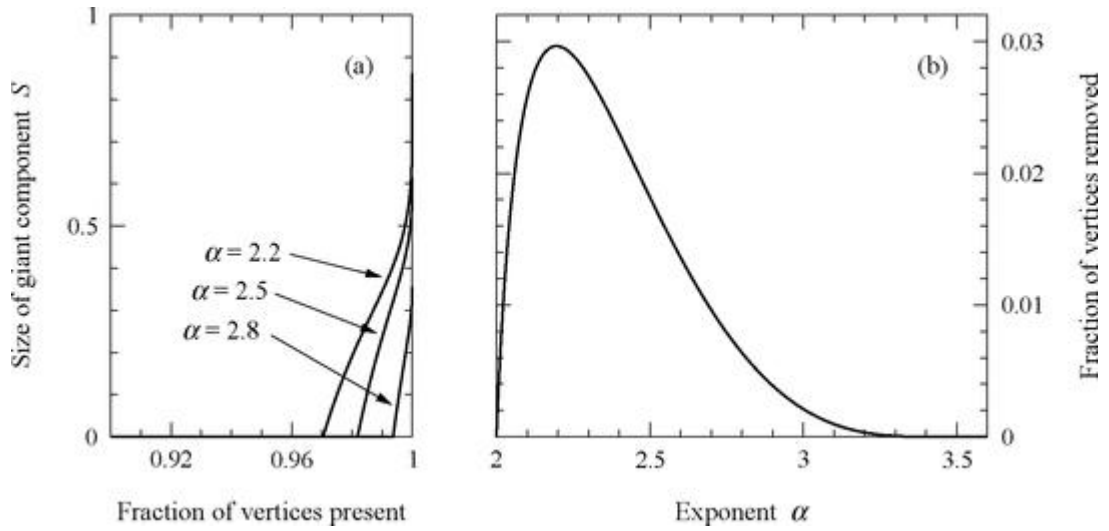



**Figure 16.7: Removal of the highest-degree vertices in a scale-free network.** (a) The size of the giant cluster in a configuration model network with a power-law degree distribution as vertices are removed in order of their degree, starting with the highest-degree vertices. Only a small fraction of the vertices need be removed to destroy the giant cluster completely. (b) The fraction of vertices that must be removed to destroy the giant cluster as a function of the exponent $\alpha$ of the power-law distribution. For no value of $\alpha$ does the fraction required exceed 3%.

These results suggest, for example, that were we able to find the highest degree vertices in a network of disease-causing contacts and vaccinate them to effectively remove them from the network, it would be a much more efficient strategy for disease control than simply vaccinating at random.

A particularly striking example of the effect described here arises in networks with power-law degree distributions. In these networks, as we have seen, uniform removal of vertices *never* destroys the giant cluster, provided the exponent of the power-law lies between two and three. By contrast, removal of the highest-degree vertices in these networks has a devastating effect. Once again we cannot solve for $S$ in closed form in the power-law case but it is reasonably straightforward to perform a numerical solution. Figure 16.7a shows the equivalent of Fig. 16.6b for the power-law case, and as we can see the giant cluster disappears extraordinarily rapidly as the high-degree vertices are removed. Only a few percent of the vertices need be removed to completely destroy the giant cluster, the exact value depending on the exponent of the power law.

Indeed, if we want to calculate only the fraction that need be removed to destroy the giant cluster, we can do so by observing once again that the phase transition at which the giant cluster appears or disappears falls at the point where the non-trivial solution of Eq. (16.33) appears or

disappears, which is the point at which the right-hand side of the equation is tangent to the line $y = u$ at $u = 1$. That is, the general criterion for the transition point is

$$f_1'(1) = 1. \tag{16.41}$$

(Alternatively, we could say that the giant cluster exists if and only if $f_1'(1) > 1$.) Again, exact solutions are often not possible but we can solve numerically. Doing this for the power-law case we find the results shown in Figure 16.7b, which plots the fraction of vertices that need be removed to destroy the giant cluster as a function of the exponent $\alpha$. As we can see, the curve peaks around $\alpha = 2.2$ at a value just below 3%. Thus in no case need we remove more than 3% to destroy the connectivity in the network.

Scale-free networks are thus paradoxically both robust and fragile, a point first emphasized by Albert *et al.* [14]. On the one hand, they are remarkably robust to the random failure of their vertices, with the giant cluster persisting no matter how many vertices we remove. (Although one should bear in mind the *proviso* of Section 16.2.1 that the size of the giant cluster matters also, and this becomes very small when the fraction $\varphi$ of occupied vertices tends to zero.) On the other hand, scale-free networks are very fragile to attacks targeted specifically at their highest-degree vertices. We need remove only the tiniest fraction of the high-degree hubs in such a network to entirely destroy the giant cluster.

The fragility of scale-free network to such targeted attack is both bad news and good news. Some networks we wish to defend against possible attack. The Internet is an example: a communication network that can easily be brought down by a malicious adversary targeting just a few of its most crucial vertices may be a disaster waiting to happen.

On the other hand, results like these could also be exploited to help eradicate or reduce disease by targeting vaccination efforts at network hubs. It is worth noting, however, that it's not necessarily easy to find the hubs in a network, so that implementation of a targeted vaccination strategy may be difficult. In most cases one does not know the entire network and so cannot simply pick out the high-degree vertices from a list.

One intriguing way of getting around this problem has been put forward by Cohen *et al.* [76], who suggest that we make use of the structure of the network itself to find the high-degree vertices. In their scheme, which they call "acquaintance immunization," they propose that one choose members of the population at random and then get each of them to nominate an acquaintance. Then that acquaintance receives a vaccination against the disease under consideration. The acquaintance in this scenario is a "vertex at the end of an edge," so in the configuration model it would have degree distributed according to the excess degree distribution, Eq. (13.46), rather than the original degree distribution of the network. But the excess degree distribution, as discussed in Section 13.3, is biased towards high-degree vertices since there are more edges that end at a high-degree vertex than at a low-degree one. Thus the selection of individuals in the scheme of Cohen *et al.* is also biased towards those with high degree. The selected individuals are not guaranteed to be the highest-degree vertices in the network, but we are a lot more likely to find the hubs this way than if we just choose vertices at random and in simulations the acquaintance immunization scheme appears to work quite well.

The acquaintance immunization scheme does have some drawbacks. First, contact networks in the real world are of course not configuration models and it is unclear how accurately the theoretical results describe real situations. Second, real contact networks mostly don't have power-law degree distributions, instead having somewhat shorter tails than the typical power law, which will reduce the effectiveness of the scheme, or indeed of any scheme based on targeting the highly connected vertices. Another issue is that, in asking people to name their acquaintances, the acquaintance immunization scheme necessarily probes the network of who is acquainted with

whom, which is in general not the same as the network of disease transmission, since people who are acquainted don't necessarily have regular physical contact of the type necessary to spread disease and because diseases can be and often are transmitted between people who don't know one another. We can do our best to make the networks similar, asking participants to name only acquaintances whom they have seen recently and in person, rather than those they might not have seen for a while or might only have to talked to on the phone. Still, the differences between the two networks means that the scheme might end up focusing vaccination efforts on the wrong set of people.

## 16.4 PERCOLATION IN REAL-WORLD NETWORKS

Having seen how percolation plays out in model networks, let us now take a look at some real ones. If we have data on the structure of a network then we can simulate the percolation process on a computer, removing vertices one by one and examining the resulting clusters. Although this is straightforward in theory, it requires some care to get good results in practice. The main issue is that the percolation process is normally a random one: the vertices are removed in random order, which means that the cluster sizes can vary depending on the precise order we choose. Even in the case where vertices are removed in decreasing order of their degree the process is still random to some extent since there can be many vertices with a given degree, among which we must choose somehow. To avoid possible biases, we usually choose among them at random.

This randomness can easily be simulated on a computer using standard random number generators, but the results of the simulation will then vary from one run of our simulation to another depending on the output of the generator. To get a reliable picture of how percolation affects a network we must perform the entire calculation many times, removing the vertices in different random orders each time, so that we can see what the typical behavior is, as well as the range of variation around that typical behavior. And this in turn means that we need to be able to perform the percolation calculation quickly. In a typical situation we might want to repeat the percolation calculation a thousand times with different random orders of removal and even if each calculation took just one minute of computer time, all thousand runs would still take a day.

If we are crafty, however, we can do much better than this and get an answer in just a few seconds for networks of the typical sizes we have been considering in this book.

## 16.5 COMPUTER ALGORITHMS FOR PERCOLATION

The simplest way to simulate the percolation process on a computer is to make use of the breadth-first search algorithm of Section 10.3.4, which can find all components in a network in time $O(m + n)$, where $m$ is the total number of edges in the network and $n$ is the total number of vertices, or just $O(n)$ for a sparse network in which $m \propto n$. If we remove a certain randomly chosen set of vertices from a network, along with the edges attached to them, then the resulting percolation clusters are by definition the components of the network that remains, and hence we can use the component-finding algorithm to find the clusters. Then we can, for example, look through those clusters until we find the largest one.

In the case of uniformly random removal of vertices, for instance, we would go through each vertex in turn, removing it (and its edges) from the network with probability $1 - \varphi$, finding the clusters, and (say) measuring the size of the largest one. Then we repeat the entire calculation, starting with the complete network again, removing a different set of vertices, and finding the clusters. Repeating the calculation a large number of times, we can calculate a mean value $S(\varphi)$ for the size of the largest component when vertices are present or functioning with probability $\varphi$.

If we are interested in only a single value of $\varphi$, this is, in fact, the best algorithm to use and the fastest known way of getting an answer. Usually, however, we are interested, as in previous sections, in the behavior of the system over the whole range of $\varphi$ from zero to one, or at least some portion of that range. In that case, we would have to repeat the whole calculation above for many values of $\varphi$ in the range of interest and this process is time-consuming and is not the best way to approach the problem.

Consider instead the following alternative approach, which appears at first to be only a slight variation on the previous one, but leads, as we will see, to much more efficient algorithms. Instead of making each vertex in the network occupied with independent probability $\varphi$, let us make a fixed number $r$ of vertices occupied, repeating the calculation many times for a given value of $r$ and averaging to get a figure $S_r$ for the size of the largest component (or any other quantity of interest) as a function of $r$.

The calculation doesn't directly give us the result we want: $S_r$ is not the same as $S(\varphi)$ and it is the latter we are interested in. If, however, we know the value of $S_r$ for every allowed value of the integer $r$, i.e., from 0 to $n$, then we can calculate $S(\varphi)$ as follows. If each vertex in the network is occupied with probability $\varphi$, then the probability that there are exactly $r$ vertices occupied is given by the binomial distribution

$$p_r = \binom{n}{r} \phi^r (1-\phi)^{n-r}.$$

(16.42)

Averaging over this distribution, the average size of the largest component as a function of $\varphi$ is then

$$S(\phi) = \sum_{r=0}^{n} p_r S_r = \sum_{r=0}^{n} \binom{n}{r} \phi^r (1-\phi)^{n-r} S_r.$$

(16.43)

At first sight, this appears to be a less promising approach for calculating $S(\varphi)$ than the previous approach. To make use of Eq. (16.43) we need to know $S_r$ for all $r$ and it takes time $O(m + n)$ to calculate $S_r$ for one value of $r$ using breadth-first search, so it is going to take $O(n(m + n))$ to calculate for all $n$ values, or $O(n^2)$ on a sparse network. Given that we also need to perform each calculation of $S_r$ many times to average over the randomness, the entire process could take a very long time to complete.

There is however a faster way to calculate $S_r$ for all $r$, inspired by the simple observation that if we have already found all the clusters in a network with $r$ vertices present, then we can find the clusters with $r + 1$ vertices simply by adding one more vertex. Most of the clusters do not change very much when we add just one vertex, and if we can find only the clusters that change upon adding a vertex, then we can save ourselves the work of performing an entire new breadth-first search, and hence save ourselves a lot of computer time. A simple algorithm for doing this works as follows.

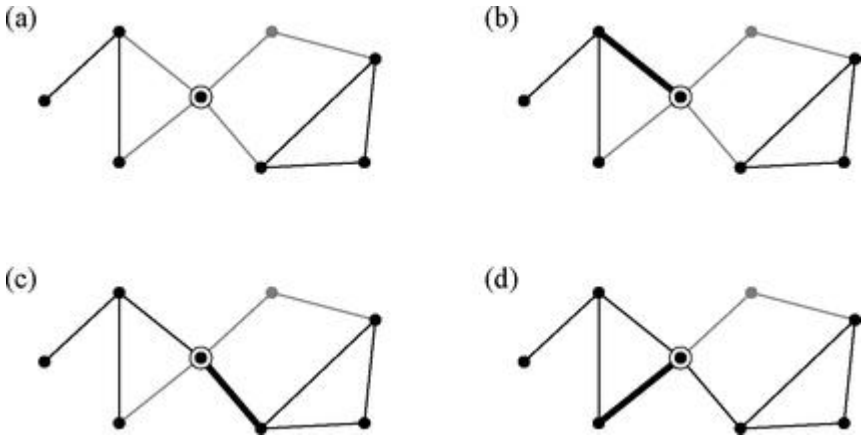



**Figure 16.8: Percolation algorithm.** In the percolation algorithm described in the text we add vertices to our network one by one, rather than taking them away. Each addition consists of several steps. (a) We add the vertex itself but none of its accompanying edges yet. At this stage the vertex constitutes a new cluster in its own right. (b) We start adding the accompanying edges (if any) in any order we like. Only edges that connect to other vertices already present in the network are added. The first edge added (if any are added) will thus, by definition, always join the new vertex to one of the previously existing clusters. Or to put it another way, it will join two clusters together, one of the old clusters and the new cluster that consists of just the single added vertex. (c) In this example the next edge added also joins two clusters together. (d) The final edge added joins two vertices that are already members of the same cluster, so the cluster structure of the network does not change.

Rather than removing vertices from the complete network, our algorithm works by building the network up from an initial state in which no vertices are occupied and switching on vertices one by

one until we recover the entire network. As we add each vertex to the network we also add the accompanying edges that join it to other vertices. Only connections to other vertices that are already present need be added.

For the purposes of our algorithm, let us break down this process as shown in Fig. 16.8. Each new vertex is first added with, initially, no accompanying edges (panel (a) in the figure). In this state it forms a cluster all on its own. Then, one by one, we add its edges, those that connect it to other vertices already present. If there are no edges attached to the vertex or none connect to vertices already present, then our new vertex remains a cluster on its own. If there are edges, however, then the first edge we add joins our vertex to an adjacent cluster—see Fig. 16.8b. Subsequent edges are more complicated. They can do one of two things. An edge can connect our vertex to another, different cluster, in which case in the process it joins two clusters together making them into a single cluster—see Fig. 16.8c. Alternatively, it could join our vertex to another member of the same cluster that it already belongs to, as in Fig. 16.8d. In this case, no clusters are joined together, and in terms of the size and identity of the clusters the added edge has no effect.

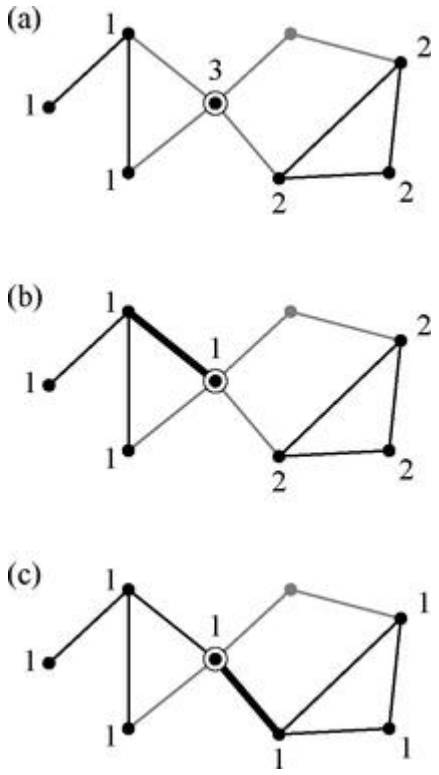



**Figure 16.9: Using labels to keep track of clusters.** In the algorithm described in the text, each vertex is given a label, typically an integer, to denote which cluster it belongs to. In this example there are initially two clusters, labeled 1 and 2. Then a new vertex is added between them. (a) The new vertex is added initially without its accompanying edges and is labeled as a new cluster, cluster 3. (b) An edge is added that connects cluster 3 to cluster 1, so we relabel one cluster to give it the same label as the other. In the algorithm described in the text we always relabel the smaller of the two clusters, which is cluster 3 in this case. (c) The next edge added joins clusters 1 and 2 and we relabel cluster 2 since it is smaller.

To keep track of the clusters in the network, therefore, our algorithm needs to do two things. First, when an edge is added it needs to identify the clusters to which the vertices at either end belong. Second, if the clusters are different, it needs to join them together into a single cluster. (If they are the same nothing need be done.)

There are various ways of achieving this but a simple one is just to put a label, such as an

integer, on each vertex denoting the cluster to which it belongs—see Fig. 16.9a. Then it is a simple matter to determine if two vertices belong to the same cluster (they do if their labels are the same), and joining two clusters together is just a matter of relabeling all the vertices in one of the clusters to match the label of the other cluster. This process is illustrated in Fig. 16.9.

Then our algorithm is as follows:

1. Start with an empty network with no occupied vertices. Let $c = 0$ be the number of clusters in the network initially. Choose at random an order in which the vertices will be added to the network.
2. Add the next vertex in the chosen order, initially with no edges. This vertex is a cluster in its own right, so increase $c$ by one and label the vertex with label $c$ to indicate which cluster it belongs to. Also make a note that cluster $c$ has size 1.
3. Go through the edges attached to this vertex one by one. For each edge determine whether the vertex at the other end has already been added to the network. If it has, add the edge to the network.
4. As each edge is added, examine the cluster labels of the vertices at either end. If they are the same, do nothing. If they are different, choose one of the clusters and relabel all its vertices to have the same label as the other cluster. Update the record of the size of the cluster to be equal to the sum of the sizes of the two clusters from which it was formed.
5. Repeat from step 2 until all vertices have been added.

At the end of this process, we have gone from an entirely empty network to the complete final network with all vertices and edges present and in between we have passed through a state with every possible intermediate number $r$ of vertices. Moreover, in each of those states we had a complete record of the identities and sizes of all the clusters which we can use, for instance, to find the size $S_r$ of the largest cluster. Then we can feed the results into Eq. (16.43) to get $S(\varphi)$ for any $\varphi$. As before, we will typically want to average the results over many runs of the algorithm to allow for random variations from one run to another, which arise from variations in the order in which the vertices are added. This, however, is no longer a serious impediment to finishing the calculation because, if implemented appropriately, the algorithm can be made to run very quickly.

The most time-consuming part of the algorithm is the relabeling of clusters when they are joined together. Note however that when an edge joins two different clusters we are free to choose which of the two we relabel. It turns out that the speed of the algorithm can be improved greatly if we choose always to relabel the smaller one. (If the two clusters have the same size, it does not matter which we choose to relabel.) To see this, consider the following argument.

If we always relabel the smaller of two clusters, then the relabeled cluster must have been joined with one at least as large as itself and hence it is now a part of a cluster at least twice its size. Thus every time a vertex is relabeled the cluster it belongs to at least doubles in size. Given that each vertex starts off as a cluster in its own right of size 1, the size of the cluster to which it belongs after $k$ relabelings is thus at least $2^k$. Since no vertex can belong to a cluster of size greater than the size $n$ of the whole network, the maximum number of relabelings a vertex can experience during the entire algorithm is given by $2^k = n$ or $k = \log_2 n$, and the maximum number of relabeling operations on all $n$ vertices is thus $n \log_2 n$. Thus the total time to perform the relabeling part of the algorithm is $O(n \log n)$.

The other parts of the algorithm are typically faster than this. The adding of the vertices takes $O(n)$ time and the adding of the edges takes $O(m)$ time, which is the same as $O(n)$ on a sparse network with $m \propto n$. So the overall running time of the algorithm to leading order is $O(m + n \log n)$, or $O(n \log n)$ on a sparse network, which is much better than our first estimate of $O(n(m + n))$ above.

Essentially the same algorithm can also be used when vertices are added or removed with probabilities other than the uniformly random ones considered here. For instance, if vertices are to be removed in decreasing order of their degrees we simply reverse that process and add vertices to an initially empty network in increasing order of degrees. The details of the algorithm itself are

unchanged—only the order of the vertices changes.

This algorithm works well in practice for almost all calculations. It is not, however, the very fastest algorithm for the percolation problem. There exists an even faster one, which runs in $O(m + n)$ time (or $O(n)$ for a sparse network) and is also considerably simpler to program, although its outward simplicity hides some subtleties. The reader interested in learning more about this approach is encouraged to look at Ref. [255].

### 16.5.1 RESULTS

Figure 16.10 shows results for four different networks as a function of the fraction of occupied vertices. In this case, the occupied vertices are chosen uniformly at random. The figure shows in each case the size $S$ of the largest cluster as a fraction of system size, plotted as a function of $\varphi$. As described in Section 16.2.1, the largest cluster acts as a proxy for the giant cluster in numerical calculations on fixed networks for which the idea of a giant cluster, as a cluster that scales with system size, is meaningless.

The top two networks in the figure, a power grid and a road network, are both networks with non-power-law degree distributions—the power grid has a roughly exponential distribution while the road network has only vertices of degree one to four and nothing else. For these cases, we expect to see behavior of the generic type described in Section 16.2.1: a continuous percolation transition at a non-zero value of $\varphi$ from a regime in which $S \simeq 0$ to a regime of non-zero $S$. Because the networks are relatively small, however (4941 vertices for the power grid, 935 for the road network), we also expect to see some rounding of the transition (see Section 16.2.1).

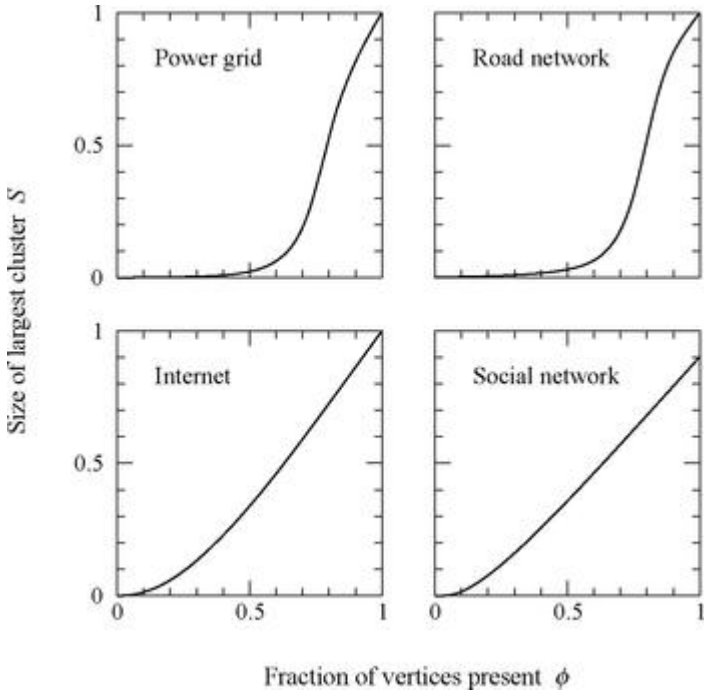



**Figure 16.10: Size of the largest percolation cluster as a function of occupation probability for four networks.** The four frames of this figure show the size of the largest cluster, measured as a fraction of network size, for random removal of vertices from four real-world networks: the western United States power grid, the network formed by the US Interstate highways, the Internet at the level of autonomous systems, and a social network of professional collaborations between physicists. Each curve is averaged over 1000 random repetitions of the calculation, which is why the curves appear smooth.

And this is in fact what we do see. In each of these two cases $S$ is close to zero below a certain value of $\varphi$, then grows rapidly but with a certain amount of rounding near the transition. Overall,

other than the rounding, the shape of the curves is qualitatively similar to that of Fig. 16.4. One could even tentatively make an estimate of the percolation transition, which appears to fall around $\varphi = 0.6$ or 0.7 in both networks.

The bottom two frames in the figure tell a different story. These show results for percolation on the Internet and a social network. Both of these networks have approximately power-law degree distributions and thus, based on the insights of Section 16.2.1, might be expected to show no percolation transition (or a transition at $\varphi = 0$ if you prefer) and non-linear growth of the largest cluster with growing $\varphi$. Again our expectations seem to be borne out, at least qualitatively, by the numerical results. In both networks the value of $S$ appears to take non-zero values for all $\varphi > 0$ and the initial growth for small $\varphi$ shows some curvature, indicating non-linear behavior.

Thus our percolation theory for random graphs seems in this case to provide a good general guide to the robustness of networks. The power-law networks are robust against random removal of vertices, in the sense that a fraction of the vertices that haven't been removed remain connected in a large cluster even when most vertices have been removed. The non-power-law networks, by contrast, become essentially disconnected after relatively few vertices have been removed—just about 40% in this case.

Figure 16.11 shows results for the same four networks when vertices are removed in order of degree, highest degrees first. As we can see, this "attack" on the network is more effective at reducing the size of the largest component than is random removal for all four networks. However, the difference between Figs. 16.10 and 16.11 is not so great for the first two networks, the power grid and the road network. The giant component in both of these networks survives nearly as long under the targeted attack as under random removal. This is as we would expect, since neither has a significant number of very high-degree vertices (the road network, with maximum degree four, has none at all), so that removal of the highest-degree vertices is not so very different from the removal of vertices of average degree.

For the second two networks, however, the Internet and the collaboration network, which both have roughly power-law degree distributions, the effect is far larger. Where these networks were more resilient to random removal than they others, they are clearly less resilient, at least by this measure, to targeted attack. The Internet in particular has a largest cluster size that falls essentially to zero when only about 5% of its highest-degree vertices have been removed, a behavior similar again to our theoretical calculations (see Fig. 16.7 on page 613). Thus the real Internet appears to show the mix of robust and fragile behavior that we saw in our calculations for the configuration model, being remarkably resilient to the random removal of vertices but far more susceptible to targeted attacks.

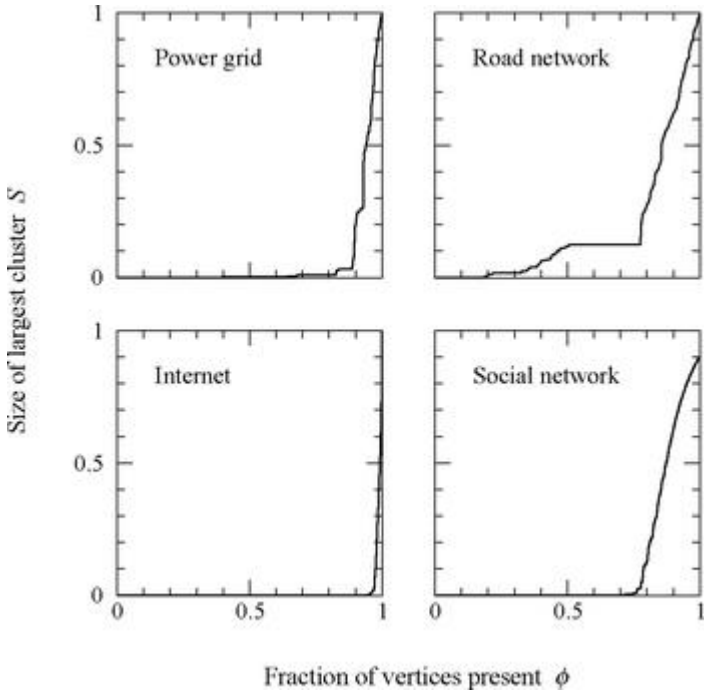


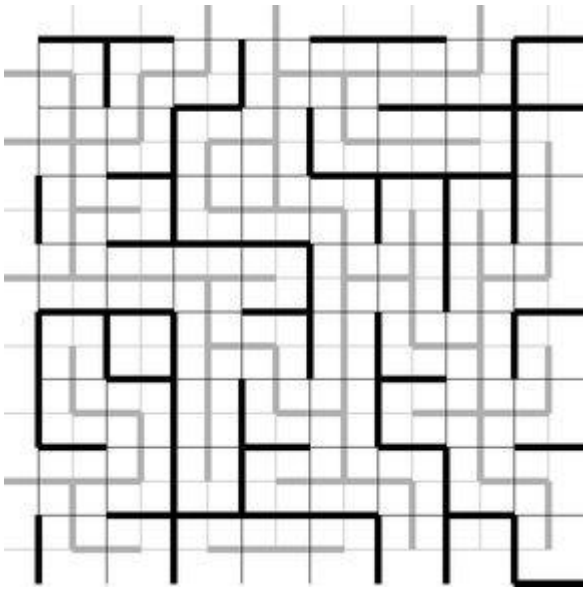
**Figure 16.11: Size of the largest percolation cluster as a function of occupation probability for targeted attacks on four networks.** The four frames in this figure show the size of the largest cluster, measured as a fraction of network size, for the same four networks as Fig. 16.10, when vertices are removed in degree order, highest-degree vertices first. Since this is mostly a deterministic process and not a random one (except for random choices between vertices of the same degree) the curves cannot be averaged as in Fig. 16.10 and so are relatively jagged.

Overall, therefore, the percolation theory seems to be successful as a qualitative guide to the resilience of networks. Certainly it does not perfectly predict the exact behavior of individual networks, but it gives a good feel for the behavior we expect of networks as vertices fail or are removed, as a function of their degree distribution.

In the next chapter we will see another application of percolation, to the spread of diseases in networks.

## PROBLEMS

**16.1** Consider the problem of bond percolation on a square lattice and consider the following construction:



Here we have taken a bond percolation system (in black) and constructed another one interlocking it (in gray), such that the bonds of the new system are occupied if and only if the intersecting bond on the old system was not. Such an interlocking system is called a *dual lattice*.

a. If the fraction of occupied bonds on the original lattice is $\varphi$, what is the fraction of occupied bonds on the dual lattice?
b. Show that there is a path from top to bottom of the dual lattice if and only if there is no path from side to side of the original lattice.
c. Hence show that the percolation transition for the square lattice occurs at $\phi = \frac{1}{2}$.

**16.2** Consider the site percolation problem with occupation probability $\varphi$ on a Poisson random graph with mean degree $c$. Let $\pi_s$ be the probability that a vertex belongs to an non-giant percolation cluster of $s$ vertices and define a generating function $h(z) = \sum_{s=0}^{\infty} \pi_s z^s$.

a. Show that $h(z) = 1 - \varphi + \varphi z e^{c[h(z)-1]}$.
b. Hence show that the mean size of a small cluster in the non-percolating regime (no giant cluster) is

$$\langle s \rangle = \frac{\phi}{1 - c\phi}.$$

c. Define $f(z) = [h(z) - 1 + \varphi]/\varphi$. Using the Lagrange inversion formula, Eq. (12.49), solve for the coefficients in the series expansion of $f(z)$ and hence show that

$$\pi_s = \begin{cases} 1-\phi & \text{if } s = 0, \\ \phi e^{-sc\phi}(sc\phi)^{s-1}/s! & \text{if } s > 0. \end{cases}$$

**16.3** Consider a configuration model network that has vertices of degree 1, 2, and 3 only, in fractions $p_1$, $p_2$, and $p_3$, respectively.

a. Find the value of the critical vertex occupation probability $\varphi_c$ at which site percolation takes place on the network.
b. Show that there is no giant cluster for any value of the occupation probability $\varphi$ if $p_1 > 3p_3$. Why does this result not depend on $p_2$?
c. Find the size of the giant cluster as a function of $\varphi$. (Hint: you may find it useful to remember that $u = 1$ is always a solution of the equation $u = 1 - \varphi + \varphi g_1(u)$.)

**16.4** In Section 16.3 we examined what happens when the highest-degree vertices are removed from a configuration model network with a power-law degree distribution $p_k = k^{-\alpha} / \zeta(\alpha)$ for $k \geq 1$ and $p_0 = 0$.

a. Show that in this case the phase transition at which the giant cluster disappears occurs when all vertices with degree $k > k_0$ have been removed, where the cut-off parameter $k_0$ satisfies

$$\sum_{k=1}^{k_0} (k^{-\alpha+2} - k^{-\alpha+1}) = \zeta(\alpha - 1).$$

b. Using the fact that $\sum_1^{k_0} k^{-x} + \sum_{k_0+1}^{\infty} k^{-x} = \zeta(x)$, and making use of the trapezoidal rule (Eq. (14.115) on page 524) for large values of $k$, show that

$$\sum_1^{k_0} k^{-x} \simeq \zeta(x) - \tfrac{1}{2}(k_0+1)^{-x} - \frac{(k_0+1)^{-x+1}}{x-1}.$$

c. Keeping leading-order terms in $k_0$ only, show that the giant cluster disappears approximately when $(k_0 + 1)^{-\alpha+3} = (\alpha - 3)[\zeta(\alpha - 2) - 2\zeta(\alpha - 1)]$.
d. Find the approximate value of $k_0$ at the point where the giant cluster disappears for $\alpha = 2.5$.

**16.5** Consider the computer algorithm for percolation described in Section 16.5, but suppose that upon the addition of an edge between two clusters we relabel not the smaller of the two clusters but one or the other chosen at random. Show by an argument analogous to the one in Section 16.5 that the worst-case running time of this algorithm is $O(n^2)$, which is substantially worse than the $O(n \log n)$ of the algorithm that always relabels the smaller cluster.